(i) → Fundamentals; Upto control statements

(ii) → Functions, pointers, Array

(iii) → MuHiDimension Array, Struct, Union,

(iv) → File Handling and Data Structure, Dyn. mem. Alloc

v) → misc (DLL, callback code, Volatile, etc ...)

Dynamic Memory Allocation

→ On demand Memory Allocation, Runtime mem. Allocation

Define the static mem Allocation.

or fixed sized mem-Allocation

or compile time mem-Allocation

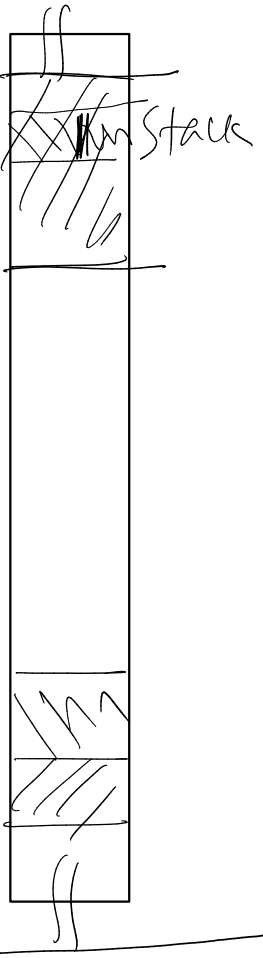
```
main(){
   int v;
   ==
sf("%d",&v);
 if(v<0)
  {
     fun();
  }
  ==
}
```

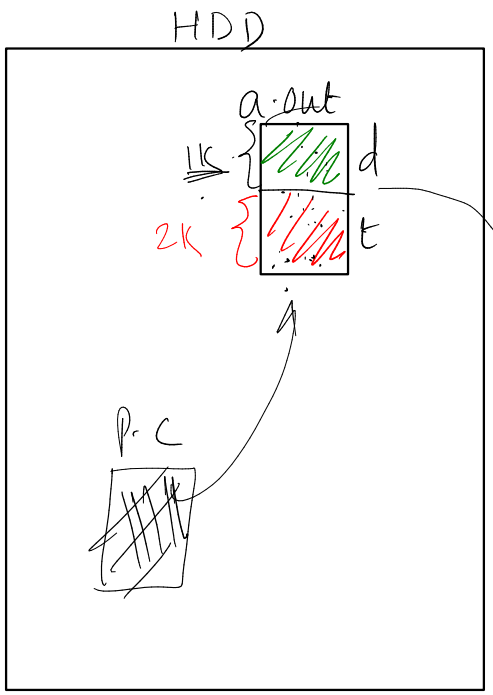
```
fun( )
{
 char arr[400];
 ==
}
```

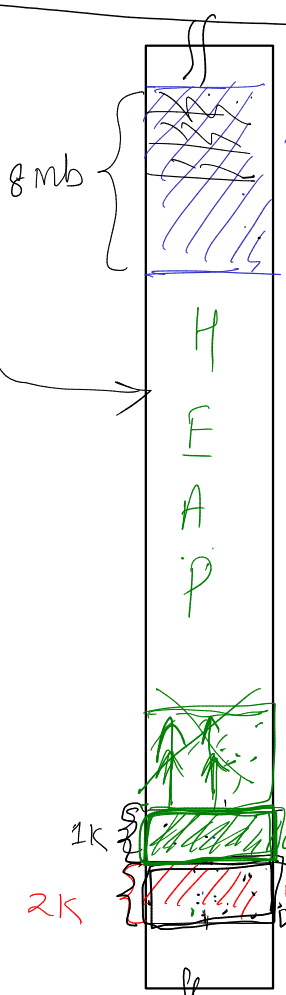
=) gcc prog.c -) d

t

=) ./a.out







CPU

malloc
calloc
realloc
free

HEAP or Extd. Data Segment

List N Prime numbers from S.

N → count

S → starting no.

N = 5

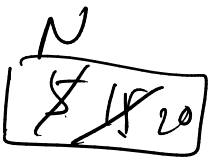
S = 10

11, 13, 17, 19, 23

---

① Store N prime nos from S, in an Array.

int arr[N];

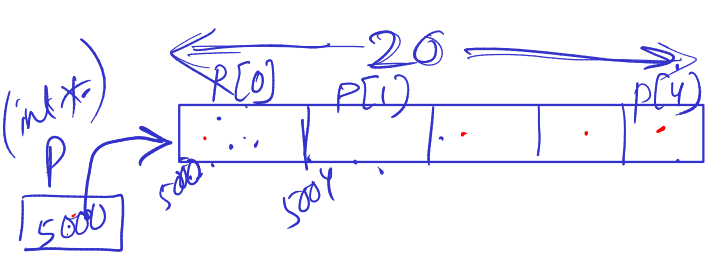


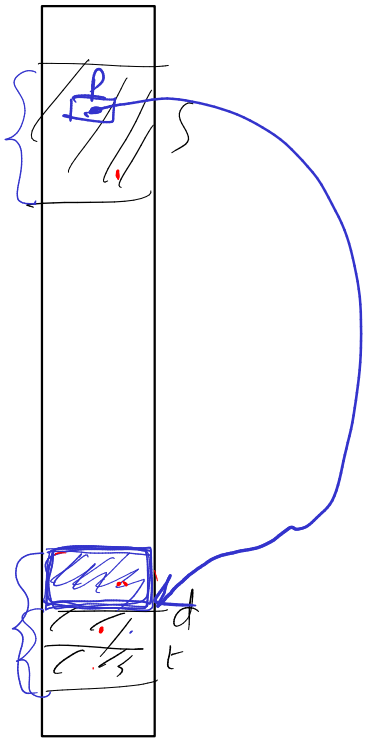
malloc / calloc ; Allocates Memory

realloc : Resize the dyn-mem-block

free : Un-reserve the dyn-allocated mem Block

```
int *p;
sf("%d", &N); // N=5
p=malloc(N*sizeof(int)); // malloc Library function, requests the mem-mgt of OS, to reserve the requsted size
if(p==NULL){
pf("Insufficient Memory"); }
```






```
void *malloc(size_t nBytes);
```

```
i=0
for(data=5 ; i<N ; data++)
 { if (checkPrime(data)){
    P[i] = data; i++; }

}

≡ //free(P);
return 0;
}
```

```
void free(void *);
```

(ii) W.A.P to store prime Numbers of a range, in an array.

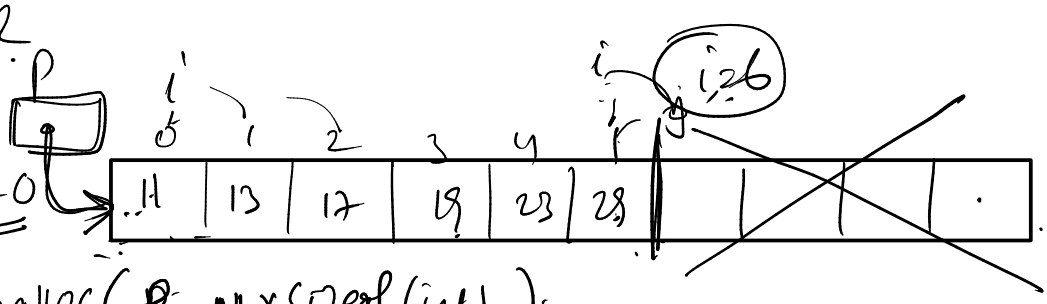
100 - 300
↳ 200

9 //

min: 10
max: 30
(max-min)/2

O/P

0	1	2	3	4	5
11	13	17	19	23	29



N = 10

N=10
i=6

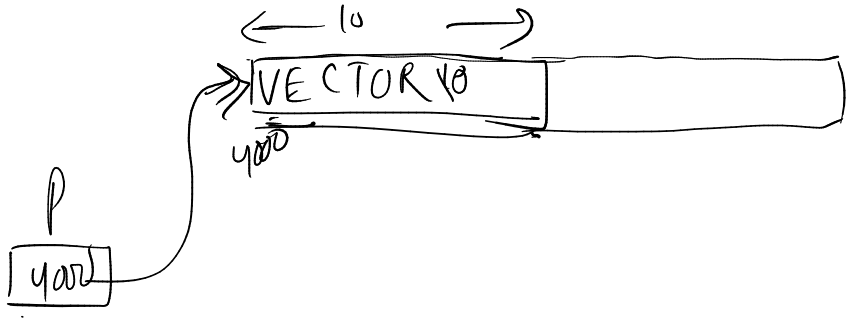
N = i
=6

realloc(P, N*sizeof(int));

NO wastage / NO shortage

```
void *realloc(void *, size_t);
```

```
char *p;
int n = 10;
≡
p = malloc(n);
```




```
strcpy(p, "VECTOR");
≡
                n = n*3; // n = 30
                p = realloc(p, n);
strcat(p, "India Private Limited");
```